Cheat Sheet: AI Agents Weather and Daily Dish

Refer to this cheat sheet to learn more about various components/methods, their descriptions, and code examples.

class WeatherAgent

Initializes the Weather Agent with OpenWeather API credentials and memory system. The agent stores the API key and base URL for making weather requests and includes memory capabilities for context-aware responses.

Code Example:

```
class WeatherAgent:
    # Initialize the WeatherAgent with the required API key
    def __init__(self, api_key):
        self.api_key = api_key  # API key for authenticating with OpenWeatherMap
        self.url = "https://api.openweathermap.org/data/2.5/weather"  # Base URL for weathe
        self.memory = Memory()  # Memory object to store past interactions or context
       # Main method to handle user queries
    def answer(self, query):
        # Extract the city name from the user's query
        city = self.extract_city(query)
        # If no city is found in the query, ask the user to specify one
        if not city:
            return "Please specify a city for weather information."
        # Fetch and return the weather information for the extracted city
        return self.get_weather(city)
```

get_weather(self, city)

Makes an HTTP GET request to the OpenWeather API with specified parameters including city name, API key, and unit system. Returns weather data in JSON format including temperature, humidity, and weather conditions.

Code Example:

```
def get_weather(self, city):
    # Define query parameters for the API request
    params = {
        "q": city,                # City name for which weather data is requested
        "appid": self.api_key,    # API key for authentication
        "units": "metric"         # Return temperature in Celsius
    }
    try:
        # Send a GET request to the OpenWeatherMap API with the given parameters
        response = requests.get(self.url, params=params)
        # Raise an exception if the HTTP request returned an error status
        response.raise_for_status()
        # Parse the JSON response into a Python dictionary
        data = response.json()
        # Format and return the weather information in a user-friendly way
        return self.format_weather_response(city, data)
    except requests.exceptions.RequestException as e:
        # Handle network errors, invalid responses, or request failures
        return f"Error fetching weather data: {str(e)}"
```

extract_city(self, query)

Extracts city names from natural language queries using pattern matching and keyword detection. Handles various query formats like "weather in Paris" or "what's the temperature in London".

Code Example:

```
def extract_city(self, query):
    # Convert the user query to lowercase for case-insensitive matching
    query_lower = query.lower()
```

```
    # Define common regex patterns to extract city names from user queries
    patterns = [
        r'weather in (\w+)',        # e.g., "weather in London"
        r'temperature in (\w+)',    # e.g., "temperature in Paris"
        r'forecast for (\w+)',      # e.g., "forecast for Tokyo"
        r'how is (\w+)',            # e.g., "how is Mumbai"
    ]
    # Iterate through each pattern and attempt to find a match in the query
    for pattern in patterns:
        match = re.search(pattern, query_lower)
        if match:
            # Return the extracted city name with the first letter capitalized
            return match.group(1).capitalize()
    # Return None if no city name is found in the query
    return None
```

format_weather_response(self, city, data)

Retrieves previous weather data from memory for contextual awareness and stores the latest weather information for future reference. Enables the agent to provide comparisons and historical context.

Code Example:

```
def format_weather_response(self, city, data):
    # Recall previous weather data
    previous = self.memory.recall(city)
    current_temp = data["main"]["temp"]
    description = data["weather"][0]["description"]
    # Store current data
    self.memory.store(city, {
        "temp": current_temp,
        "description": description,
        "timestamp": time.time()
    })
    response = (
        f"The current weather in {city} is {description} "
        f"with a temperature of {current_temp}°C."
    )
    if previous:
        temp_change = current_temp - previous["temp"]
        response += f" (Temperature change: {temp_change:+.1f}°C)"
    return response
```

class Memory

Simple in-memory storage system for agent context. Stores and retrieves data using key-value pairs, allowing agents to maintain state across multiple interactions.

Code Example:

```
class Memory:
    # Initialize the memory storage as an empty dictionary
    def __init__(self):
        self.storage = {}  # Dictionary to store key-value pairs
    # Store a value in memory using a specified key
    def store(self, key, value):
        self.storage[key] = value
    # Retrieve a value from memory using the key
    # Returns None if the key does not exist
    def recall(self, key):
        return self.storage.get(key, None)
    # Clear memory contents
    # If a key is provided, remove only that entry
    # If no key is provided, clear all stored data
    def clear(self, key=None):
        if key:
            self.storage.pop(key, None)
        else:
            self.storage.clear()
```

DailyDishAgent

Initializes the FAQ-based Daily Dish Agent with a TF-IDF vectorizer for semantic similarity matching. Stores predefined questions and answers about restaurant information, menu items, and services.

Code Example:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
class DailyDishAgent:
    # Initialize the agent with a list of questions and corresponding answers
    def __init__(self, questions, answers):
        self.questions = questions  # List of sample or known questions
```

```
        self.answers = answers      # List of answers mapped to the questions
        # Initialize the TF-IDF vectorizer to convert text into numerical vectors
        self.vectorizer = TfidfVectorizer(
            stop_words="english",    # Remove common English stop words
            ngram_range=(1, 2),       # Use unigrams and bigrams for better context
            max_features=1000         # Limit vocabulary size to top 1000 features
        )
        # Convert all questions into TF-IDF vectors for similarity comparison
        self.doc_vectors = self.vectorizer.fit_transform(questions)


```

answer(self, query)

Converts text questions into numerical vectors using Term Frequency-Inverse Document Frequency (TF-IDF). This enables semantic similarity comparison between user queries and stored FAQ questions.

Code Example:

```
def answer(self, query):
    # Convert the user query into a TF-IDF vector
    # The query must be wrapped in a list as the vectorizer expects an iterable
    query_vector = self.vectorizer.transform([query])
    # Calculate cosine similarity between the query vector
    # and all stored question vectors
    similarities = cosine_similarity(
        query_vector,
        self.doc_vectors
    )[0]  # Extract similarity scores as a 1D array
    # Identify the index of the most similar question
    best_idx = similarities.argmax()
    # Retrieve the highest similarity score
    best_score = similarities[best_idx]
    # If the similarity score exceeds the predefined threshold,
    # return the corresponding answer
    if best_score >= 0.08:
        return self.answers[best_idx]
    else:
        # Fallback response when no good match is found
        return "I don't have information about that. Please contact us directly."
```

find_best_match(self, query, threshold=0.08)

Compares user queries against stored questions using cosine similarity. Returns the most relevant answer when similarity score exceeds a defined threshold, ensuring accurate and relevant responses.

Code Example:

```
def find_best_match(self, query, threshold=0.08):
    # Convert the user query into a TF-IDF vector
    query_vector = self.vectorizer.transform([query])
    # Compute cosine similarity between the query vector
    # and all stored question vectors
    similarities = cosine_similarity(query_vector, self.doc_vectors)[0]
    # Identify the indices of the top 3 most similar questions
    top_indices = similarities.argsort()[-3:][::-1]
    # Retrieve the similarity scores for the top matches
    top_scores = similarities[top_indices]
    results = []  # List to store matching results
    # Iterate through the top matches and filter by similarity threshold
    for idx, score in zip(top_indices, top_scores):
        if score >= threshold:
            results.append({
                "question": self.questions[idx],  # Matched question text
                "answer": self.answers[idx],      # Corresponding answer
                "score": score                    # Similarity score
            })
    # Return the list of best matches (can be empty if no match meets the threshold)
    return results
```

FAQ data structure

Defines the questions and answers for the Daily Dish restaurant. This structured data enables the agent to respond to common customer inquiries about hours, menu, reservations, and policies.

Code Example:

```
# List of frequently asked customer questions
# These questions will be used as reference data for matching user queries
questions = [
    "What are your opening hours?",
    "Do you take reservations?",
```

```
    "What type of cuisine do you serve?",
    "Do you have vegetarian options?",
    "Where are you located?",
    "Do you offer delivery?",
    "What is your phone number?",
    "Do you have gluten-free options?"
]
# Corresponding answers to each question above
# The index of each answer matches the index of its related question
answers = [
    "We're open Monday-Thursday 11am-10pm, Friday-Saturday 11am-11pm, Sunday 10am-9pm.",
    "Yes, we accept reservations. Call us at (555) 123-4567 or book online.",
    "We serve contemporary American cuisine with seasonal ingredients.",
    "Yes, we have several vegetarian and vegan options on our menu.",
    "We're located at 123 Main Street, Downtown.",
    "Yes, we partner with major delivery services including DoorDash and Uber Eats.",
    "You can reach us at (555) 123-4567.",
    "Yes, we offer gluten-free bread and pasta options."
]
```

AgentRouter

Creates a router that directs user queries to the appropriate specialized agent based on query content. Uses keyword matching to determine whether to route to Weather Agent or Daily Dish Agent.

Code Example:

```
class AgentRouter:
    # Initialize the router with different agent instances
    def __init__(self, weather_agent, daily_dish_agent):
        self.weather_agent = weather_agent      # Agent responsible for weather-related quer
        self.daily_dish_agent = daily_dish_agent  # Agent responsible for restaurant/FAQ qu
        # Define keywords used to identify weather-related queries
        self.weather_keywords = [
            "weather", "temperature", "forecast",
            "rain", "sunny", "climate", "humidity",
            "hot", "cold", "warm"
        ]
    # Route the user query to the appropriate agent
    def route(self, query):
        # Convert the query to lowercase for case-insensitive matching
        query_lower = query.lower()
        # Check if the query contains any weather-related keywords
        for keyword in self.weather_keywords:
            if keyword in query_lower:
                return "weather"  # Route to the WeatherAgent
        # Default route if no weather keywords are found
        return "daily_dish"  # Route to the DailyDishAgent
```

Keyword-based routing

Analyzes user queries for specific keywords to determine the appropriate agent. Weather-related keywords route to the Weather Agent, while all other queries default to the Daily Dish Agent.

Code Example:

```
def answer(self, query):
    # Determine which agent should handle the query
    route = self.route(query)
    # Route to appropriate agent
    if route == "weather":
        return self.weather_agent.answer(query)
    else:
        return self.daily_dish_agent.answer(query)
```

Multi-agent orchestration

Coordinates multiple specialized agents to handle diverse user queries. The router examines query content and delegates to the most appropriate agent, creating a unified conversational interface.

Code Example:

```
# Initialize agents
weather_agent = WeatherAgent(api_key="your_api_key_here")
daily_dish_agent = DailyDishAgent(questions, answers)
# Create router
router = AgentRouter(weather_agent, daily_dish_agent)
# Handle queries
queries = [
    "What's the weather in Paris?",
    "Do you have vegetarian options?",
    "Is it going to rain in London?",
    "What are your opening hours?"
```

```
]
for query in queries:
    response = router.answer(query)
    print(f"Q: {query}")
    print(f"A: {response}\n")
```

route_with_confidence(self, query)

Advanced routing mechanism that calculates confidence scores for each potential agent based on keyword matches and query characteristics. Routes to the agent with highest confidence.

Code Example:

```
def route_with_confidence(self, query):
    query_lower = query.lower()
    scores = {"weather": 0, "daily_dish": 0}
    # Calculate weather score
    for keyword in self.weather_keywords:
        if keyword in query_lower:
            scores["weather"] += 1
    # Calculate restaurant score
    restaurant_keywords = [
        "menu", "food", "reservation", "hours",
        "restaurant", "dish", "eat", "dining"
    ]
    for keyword in restaurant_keywords:
        if keyword in query_lower:
            scores["daily_dish"] += 1
    # Return agent with highest score
    if scores["weather"] > scores["daily_dish"]:
        return "weather", scores["weather"]
    else:
        return "daily_dish", scores["daily_dish"]
```

answer_with_fallback(self, query)

Implements comprehensive error handling to gracefully manage API failures, network issues, and invalid queries. Ensures the agent provides helpful feedback even when errors occur.

Code Example:

```
def answer_with_fallback(self, query):
    try:
        route = self.route(query)
        if route == "weather":
            response = self.weather_agent.answer(query)
        else:
            response = self.daily_dish_agent.answer(query)
        # Check if response is valid
        if not response or response.startswith("Error"):
            return self.fallback_response(query)
        return response
    except Exception as e:
        return f"I encountered an issue: {str(e)}. Please try again."
def fallback_response(self, query):
    return (
        "I'm having trouble answering that right now. "
        "Please try rephrasing your question or contact us directly."
    )
```

MonitoredRouter(AgentRouter)

Tracks agent interactions including queries, responses, routing decisions, and errors. Enables performance monitoring and debugging of agent behavior.

Code Example:

```
import logging
from datetime import datetime
class MonitoredRouter(AgentRouter):
    # Initialize the monitored router with agent instances
    def __init__(self, weather_agent, daily_dish_agent):
        # Call the parent AgentRouter initializer
        super().__init__(weather_agent, daily_dish_agent)
        self.logs = []  # Store interaction logs in memory
        # Configure the logging system
        logging.basicConfig(level=logging.INFO)
        self.logger = logging.getLogger(__name__)
    # Handle a user query with routing and monitoring
    def answer(self, query):
        # Record the start time of the request
        start_time = datetime.now()
```

```
        # Determine which agent should handle the query
        route = self.route(query)
        # Get the response from the appropriate agent via the parent class
        response = super().answer(query)
        # Create a structured log entry for this interaction
        log_entry = {
            "timestamp": start_time.isoformat(),          # Time the request was received
            "query": query,                               # User's input query
            "route": route,                               # Selected agent route
            "response": response,                         # Agent response
            "duration": (datetime.now() - start_time).total_seconds()  # Processing time in
        }
        # Save the log entry for later analysis or auditing
        self.logs.append(log_entry)
        # Write a concise log message to the application logs
        self.logger.info(f"Query routed to {route}: {query[:50]}...")
        # Return the final response to the user
        return response
```

test_agents()

Validates agent functionality by testing various query types and verifying responses. Ensures agents handle expected inputs correctly and gracefully manage edge cases.

Code Example:

```
def test_agents():
    # Test data
    test_cases = [
        {
            "query": "What's the weather in Tokyo?",
            "expected_agent": "weather",
            "should_contain": ["Tokyo", "temperature"]
        },
        {
            "query": "Do you have vegan options?",
            "expected_agent": "daily_dish",
            "should_contain": ["vegetarian", "vegan"]
        }
    ]
    # Run tests
    for test in test_cases:
        response = router.answer(test["query"])
        route = router.route(test["query"])
        # Verify routing
        assert route == test["expected_agent"], \
            f"Expected {test['expected_agent']}, got {route}"
        # Verify response content
        for keyword in test["should_contain"]:
            assert keyword.lower() in response.lower(), \
                f"Response missing expected keyword: {keyword}"
        print(f"✓ Test passed: {test['query']}")
```

Key Concepts

- **Agent:** An autonomous system that perceives its environment through sensors and acts upon that environment to achieve specific goals.
- **Routing:** The process of directing user queries to the most appropriate specialized agent based on query content and context.
- **Memory:** Storage mechanism that allows agents to maintain context and state across multiple interactions.
- **TF-IDF:** Term Frequency-Inverse Document Frequency, a numerical statistic that reflects word importance in a document collection.
- **Cosine Similarity:** A metric used to measure how similar two vectors are, commonly used for text similarity comparisons.
- **Semantic Matching:** Finding the meaning-based similarity between texts, rather than exact keyword matches.